How WheelNet Works

A step-by-step guide to the architecture

Companion to *Will It Wheel? Predicting Which Passed Cards Come Back*

October 2026

The paper's Figure 1 squeezes the whole model into five boxes. This guide unpacks them one at a time. It assumes you know how a booster draft works and roughly what a neural network is (a function with lots of tunable numbers, fitted by gradient descent). It does not assume you know transformers. Every claim about the model refers to the code in `src/openlane/nn.py`, and the sizes are those of the shipped MSH model (`results/MSH.PremierDraft/models/wheelnet_time.pt`). WheelNet is a sibling of PickNet, and this guide follows the PickNet guide's structure; Section 13 lists the differences.

Contents

1 The job, and the big picture **2**

2 What one training example looks like **2**

3 Step 1: turning a card into a vector **4**

4 Step 2: building one history token per pick **5**
4.1 "What the others took" 6
4.2 Draft context 6

5 Step 3: the history encoder **7**
5.1 Attention in one paragraph 7
5.2 The causal mask 7

6 Step 4: passed-card tokens **8**

7 Step 5: the pack decoder **8**
7.1 Two masks 9

8 Step 6: from scores to probabilities **9**

9 The features-only variant: one model for many sets **10**

10 Training **11**

11 Shapes and sizes **11**

12 Putting it together: one wheel pick, start to finish **12**

13 How WheelNet differs from PickNet **12**

14 FAQ **13**

1 The job, and the big picture

Every time a drafter passes a pack early in a round, WheelNet looks at everything they could have known at that moment and outputs, for *each* card they passed, a probability that it comes back eight picks later: "Hex Magic 88%, Super Suit 50%, Jessica Jones 7%, …". It is trained to make those probabilities honest: of the cards it gives 30%, about 30% should come back.

The model has two halves, and almost everything else is plumbing around them:

1. **The history encoder** reads the draft so far, one token per pick, and produces a *state* for each pick: a list of 192 numbers that summarises "what this drafter has seen and done so far": the packs they were shown, the cards they took, and what the rest of the table took from the packs that came back.
2. **The pack decoder** takes the cards the drafter just passed, lets them look at each other and at the history, and gives each card a score. A sigmoid turns each score into a probability.

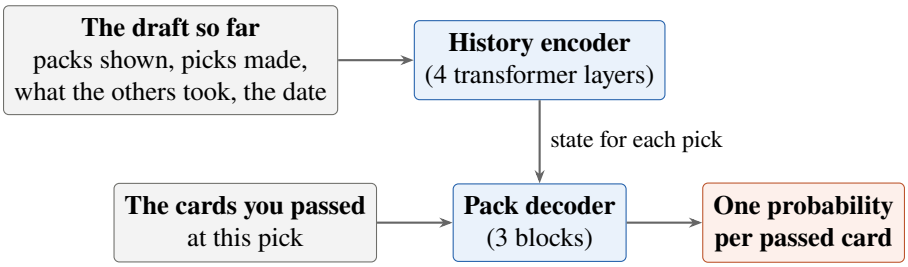


Figure 1: The model at the coarsest level.

A useful way to think about it: the history encoder answers "what do I know about this table right now?" and the pack decoder answers "given that, which of the cards I just passed will the next seven drafters leave alone, considering what else they could take instead?".

Figure 2 (page 3) is the same model with nothing left out: every layer that has trained weights, how many weights each one has, and the shape of every tensor for one draft. **Blue** boxes have trained weights; the small blue number is how many. **Grey** boxes are data, **white** boxes are fixed operations with no weights, $\oplus$ adds vectors, and ♦ marks a lookup in the card table. Shapes are per draft (a batch adds a leading B) for MSH's 14-card packs. The history-encoder layer is stacked 4 times and the pack-decoder block 3 times, each copy with its own weights; the thin grey lines that skip around each attention and MLP are the residual connections. Every decoder block reads the same history states **h**. Dropout (0.2) after each attention and MLP is not drawn. The counts add up to 3,864,001. The rest of this guide walks through the figure from top to bottom.

2 What one training example looks like

One training example is one *whole draft* by one drafter, not one passed card. With n-card packs a draft has $3n$ pick slots; for MSH's 14-card packs that is **42 slots**, numbered $t = 0, \ldots, 41$.

Which picks can wheel. A pack you see at pick p of a round comes back at pick $p + 8$, after the other seven drafters have each taken one card. It only comes back with something in it if $p + 8 < n$, so only picks $p = 0, \ldots, n - 9$ are *wheel-eligible*: 6 per pack for 14-card packs, 18 per draft. At each of those 18 picks the drafter passes up to 13 distinct cards (the pack minus their pick), so one draft holds up to $18 \times 13 = 234$ passed-card slots.

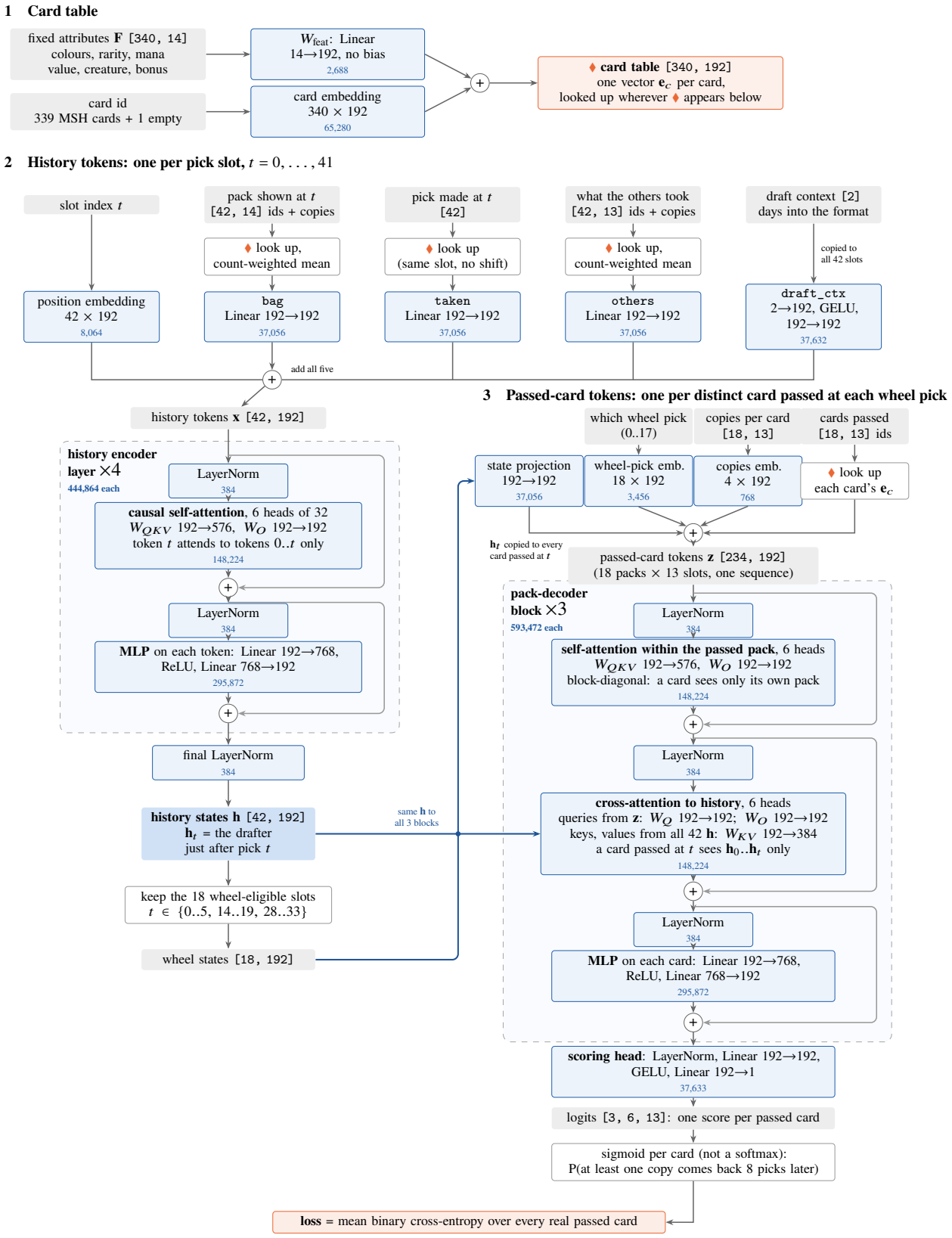


Figure 2: Every learned layer of WheelNet, for one MSH draft. Blue: trained weights (with counts). Grey: data. White: fixed operations. ♦: card-table lookup.

The label. A passed card *wheels* if at least one copy of it is in the pack the drafter is shown at $p+8$. 17Lands records that pack, so the label is free: no annotation, just a lookup eight rows later in the same draft. The returning pack must be a subset of what was passed; the code checks this and clips the handful of malformed records where it isn't (`wheel_tensors()` in `features.py`).

For each draft the data holds:

name in code	shape per draft	meaning
`pack`, `pack_n`	`[42, 14]`	the pack shown at t: up to 14 distinct card ids, and how many copies of each
`pick`	`[42]`	the card the drafter took at t
`oth`, `oth_n`	`[42, 13]`	the cards the other seven took from the pack that just came back (empty for the first 8 picks of each pack)
`idx`, `copies`	`[3, 6, 13]`	the distinct cards passed at each wheel-eligible pick, and their copy counts
`label`	`[3, 6, 13]`	did at least one copy come back? (the answer)
`mask`	`[3, 6, 13]`	false for empty slots, so they don't count
`ctx`	`[2]`	draft context: days into the format (Section 4.2)

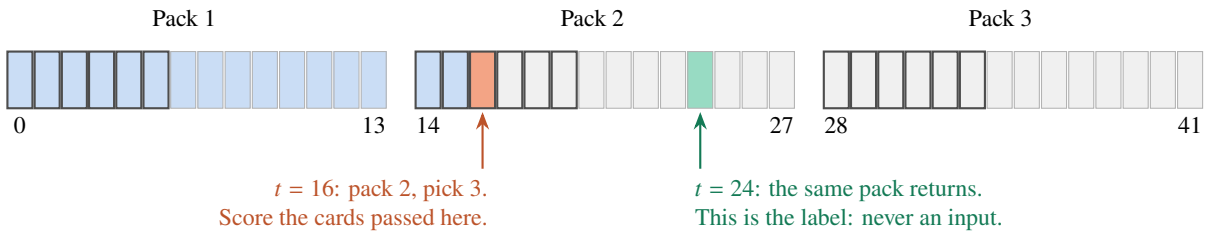


Figure 3: The 42 slots of one draft. Thick outlines mark the 18 wheel-eligible picks. When scoring the cards passed at $t = 16$ (orange), the model may use every slot up to and including 16 (blue): every pack shown and every pick made, *including the pick at 16*, since the drafter has already picked when they pass. The pack shown at $t = 24$ (green) is what the model is predicting, so it and everything else after 16 (grey) must be hidden.

The model processes 128 drafts at a time (a *batch*), so in the code most tensors have a leading batch dimension $B = 128$: `pack` is `[B, 42, 14]`, and so on. Below we usually drop B and talk about one draft.

One forward pass scores all 234 slots at once. The model does not run 18 separate times, once per wheel pick. It reads the whole draft in one go and outputs a probability for every passed card at every wheel-eligible pick. It can do this without cheating because of two masks (Sections 5 and 7): the computation for a card passed at t is wired so that it only uses slots $\leq t$. This matters more than usual here, because the answer for pick t is sitting in the data at slot $t + 8$.

3 Step 1: turning a card into a vector

Neural networks work on lists of numbers, so the first job is to give every card a vector of $d = 192$ numbers. Think of these as coordinates in a "card space" where cards the table treats alike land near each other. A card's vector is the sum of two parts:

1. **A learned vector for the card.** A lookup table with one row of 192 numbers per card (339 for MSH, plus an empty row). The numbers start random and are tuned during training. Nothing tells the model

what they mean; it discovers, say, that one direction means "everyone takes this early" because that helps it predict what wheels.

2. **A projection of fixed card attributes.** Each card also has 14 fixed numbers: five colour flags, a one-hot rarity, mana value (scaled to 0–1), a creature flag and a bonus-sheet flag. A learned 14×192 matrix maps these into the same space, so colour and rarity information is there from the first training step.

$$\mathbf{e}_c = \underbrace{\text{CardEmbedding}[c]}_{\text{learned, per card}} + \underbrace{W_{\text{feat}}\,\mathbf{f}_c}_{\text{learned map of fixed attributes}} \qquad \mathbf{e}_c \in \mathbb{R}^{192}$$

The whole table, `[340, 192]`, is computed once per forward pass and then looked up everywhere a card appears. In the code this is `card_table()`. Section 9 describes a variant that drops the learned row and builds each card from a much richer feature vector, so one model can cover many sets.

A "bag" of cards. Some inputs are a set of cards rather than one card: the pack shown, the cards the others took. The model turns a set into one vector by taking the *count-weighted average* of the card vectors, then passing it through a learned 192×192 linear layer. This is what the code calls `_bag()`. A pack with two copies of card A and one of card B becomes $\frac{2}{3}\mathbf{e}_A + \frac{1}{3}\mathbf{e}_B$. Averaging throws away order and exact composition, but keeps the gist: "this pack is mostly blue, with one strong rare". Both bags use the same card table (`tied_bag=True`), each through its own linear layer.

4 Step 2: building one history token per pick

The history encoder reads a sequence of 42 *tokens*, one per pick slot. Token t is the *sum* of five pieces of information, each first mapped into the same 192-number space:

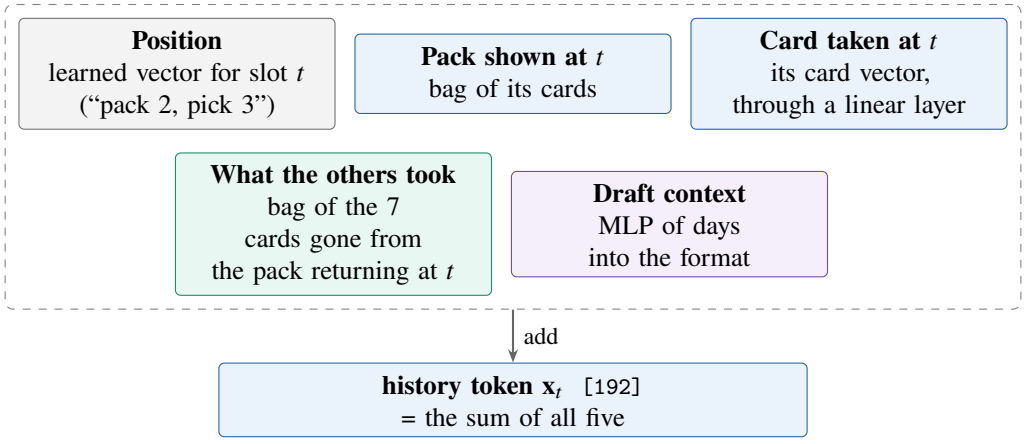


Figure 4: The five ingredients of history token t. Each is mapped to 192 numbers and they are simply added. Adding (rather than concatenating) is standard in transformers: the network learns to keep the ingredients in roughly separate directions of the space.

- **Position.** One learned vector per slot (42 of them). Without it the model would not know whether a token is pick 1 or pick 14, and wheeling depends heavily on that: a card passed at pick 1 has to survive seven drafters choosing from a full pack, one passed at pick 6 only from a nearly empty one.

- **Pack shown at t.** The bag of the cards in front of the drafter.
- **Card taken at t.** The vector of the card the drafter took *at this same pick*. This is the most important difference from PickNet, where token t holds the pick at $t-1$ because pick t is the answer. In WheelNet the question is asked after the drafter has picked, about what they passed, so their own pick is legitimately known. (The passed cards are the pack shown minus this pick.)
- **What the others took.** Explained below; this is the input that lets the model read signals.
- **Draft context.** Section 4.2.

There is no set embedding: a per-set WheelNet only ever sees one set.

4.1 "What the others took"

With eight drafters, a pack you see at pick p comes back to you at pick $p+8$, after each of the other seven has taken one card from it. Comparing the two snapshots tells you exactly which seven cards the table took, even though the data never says who the other drafters are.

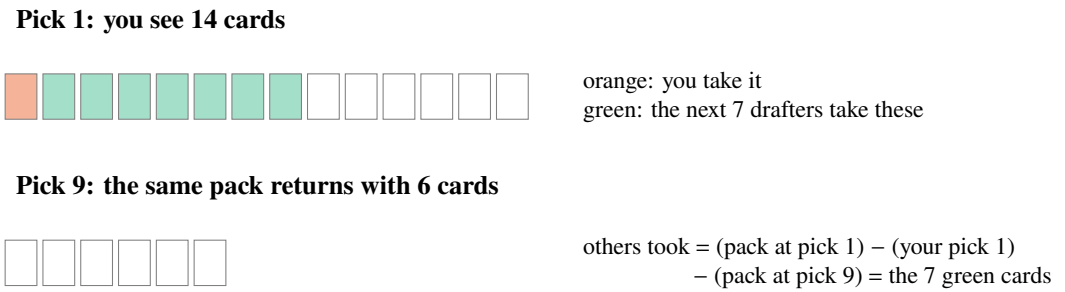


Figure 5: How "what the others took" is computed (`taken_by_others()` in `mtgdata.encode`). The same subtraction works for every pick from 9 onward in each pack; for picks 1–8 nothing has come back yet, so this input is zero. The white cards at pick 9 are exactly the cards that wheeled from pick 1.

This input is the wheel label of an earlier pick, turned around: token 8 holds the answer to the question asked at token 0. That is fine, and useful: by the time the model scores pick 2 of pack 2, it knows what wheeled (and what was taken) from every pack of round 1. If the green cards you passed are all gone but the blue ones wheel, blue is probably open, and blue cards you pass now are more likely to come back. But it is also why the masks matter so much: a token from 8 or more slots after the pick being scored would contain its answer.

4.2 Draft context

The shipped model gets two numbers per draft, identical for all 42 tokens: the number of days since the set's first draft in the data, divided by 30, and $\ln(1+\text{days})/\ln 31$. A small MLP ($2 \to 192 \to 192$) turns them into a vector added to every token.

Why the date? Early in a format, drafters haven't settled on which cards are good, so what the table takes, and so what wheels, keeps moving for weeks. An input ablation on MSH tried each optional input: days into the format was the only one that helped (validation log loss -0.0039). The drafter's rank and 17Lands skill buckets added nothing, because the drafter's picks already show how they draft, and card types were a wash. Time is a continuous number rather than a "which week" lookup because the test drafts come from the end of the format, in weeks no training draft covers.

5 Step 3: the history encoder

Now we have 42 tokens, but each one only knows about its own slot. Token 16 holds pack 16 and pick 16, but not pick 3. The history encoder's job is to let every token gather information from the earlier ones, so that the output at slot t summarises the *whole* draft up to t. It does this with *self-attention*.

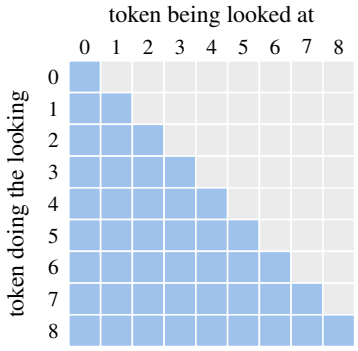
5.1 Attention in one paragraph

Each token computes three vectors from itself: a *query* ("what am I looking for?"), a *key* ("what do I contain?") and a *value* ("what do I pass on if you pick me?"). Token t compares its query with every other token's key; the better the match, the bigger the weight. It then takes the weighted average of the values and adds it to itself. So token 16 might learn to look hard at the tokens from round 1 where blue cards wheeled, and pull in "blue is flowing" from them. The query, key and value maps are learned. WheelNet uses 6 *heads*: 6 independent attention patterns run side by side, each on a 32-number slice of the vector.

A transformer *layer* is one attention step followed by a small per-token MLP ($192 \to 768 \to 192$, ReLU), each wrapped with layer normalisation before it ("pre-norm") and a residual ("add the result back to the input") connection. The history encoder stacks **4** of these layers (`nn.TransformerEncoder`).

5.2 The causal mask

If token 16 could attend to token 24, it would see the pack that returns at 24: the answer. The *causal mask* forbids that: token t may attend only to tokens $0, \ldots, t$.



Blue: allowed. Grey: masked out.

Token t contains pack t and pick t. Attending to tokens $0..t$ therefore gives it **every pack and every pick up to and including the current one**: exactly what the drafter knows at the moment they pass.

(The real mask is 42×42; this shows the first 9 slots.)

Figure 6: The causal mask (`self.causal`). Because of it, one pass over the whole draft gives 42 honest states, each blind to everything after its own slot.

The output is one *state* $\mathbf{h}_t$ (192 numbers) per slot, after a final layer normalisation. $\mathbf{h}_t$ is the model's summary of the draft just after pick t. Because of the mask, it is the same whether you compute it from the full draft or only the first $t+1$ slots; that is what makes the parallel training legitimate, and it is how you use the model live: feed in the draft so far, read off the last slot.

Only 18 of the 42 states are used to build passed-card tokens: those at the wheel-eligible slots $t \in \{0, \ldots, 5,\ 14, \ldots, 19,\ 28, \ldots, 33\}$ (`self.wheel_pos`). All 42 are available to the decoder's cross-attention, subject to its own mask.

6 Step 4: passed-card tokens

The history encoder knows the table; now the model must score the cards the drafter passed. Each distinct card c passed at wheel pick t gets its own *passed-card token*, again a sum:

$$\mathbf{z}_{t,c} = \underbrace{\mathbf{e}_c}_{\text{card vector}} + \underbrace{\text{CopiesEmbedding}[\text{copies}_c]}_{\text{1, 2 or 3+ copies}} + \underbrace{\text{WheelPickEmbedding}[t]}_{\text{which of the 18}} + \underbrace{W_{\text{state}}\,\mathbf{h}_t}_{\text{the table right now}}$$

So the token "Super Suit, passed at pack 1 pick 2" starts out knowing what Super Suit is, that it was passed at the second pick of round 1, and what the draft looks like at that moment. A pack with two copies of a card gives *one* token with copies = 2, matching the label ("at least one copy comes back"). The wheel-pick embedding is a separate table from the 42 history positions; it tells the decoder how many cards remain to absorb the seven picks.

Every wheel pick gets up to 13 passed-card slots, so there are $18 \times 13 = 234$ tokens per draft, `[234, 192]`. Empty slots are masked out. In the code the projection W_{state} is called `self.ctx`; despite the name it projects the history state, not the draft context.

7 Step 5: the pack decoder

The pack decoder refines the passed-card tokens with **3 blocks** (`nn.TransformerDecoder`). Each block does three things, each with layer normalisation before it and a residual connection around it:

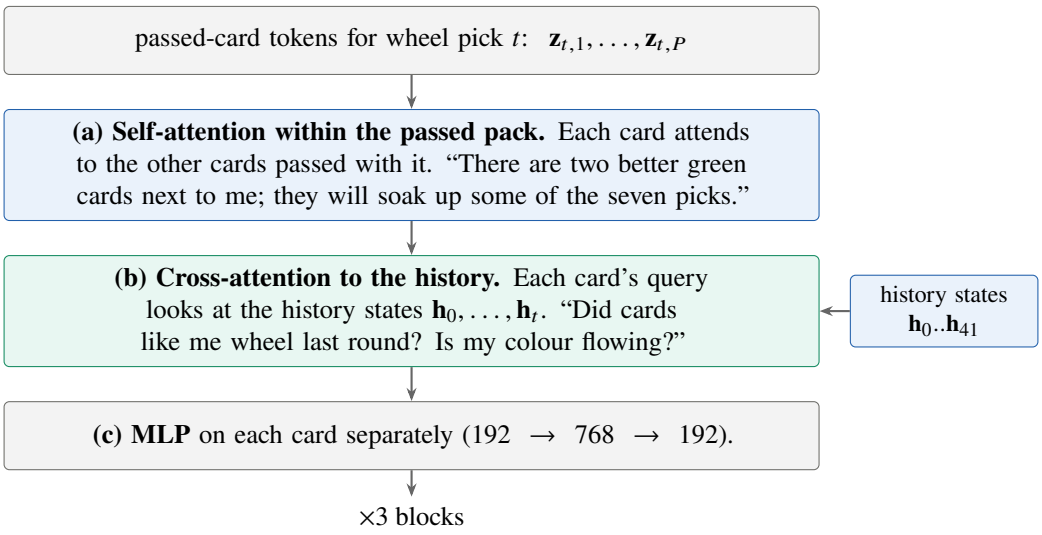


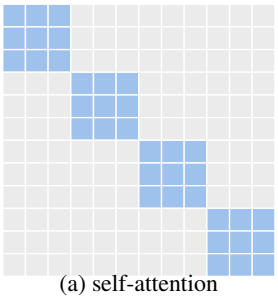
Figure 7: One pack-decoder block.

(a) is why WheelNet beats a per-card statistic. A card wheels only if the seven drafters after you all find something they want more. Whether they do depends on what else is in the pack. A decent card passed alongside two bombs is shielded; the same card as the best thing left is not. ALSA averages this away; the gradient-boosted baseline in the paper encodes it by hand as `n_better` (how many passed cards usually go before this one). Self-attention among the passed cards learns it.

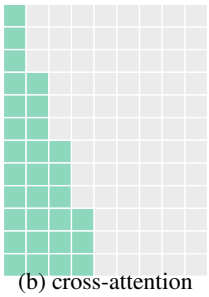
(b) is how it reads signals. Here the *passed cards* ask the questions and the *history states* answer. A card passed at pack 2, pick 3 can pull from any $\mathbf{h}_s$ with $s \leq 16$, including the round-1 states that saw what came back.

7.1 Two masks

All 18 passed packs of a draft are laid out as one sequence of 234 tokens, so the history is projected into keys and values once per draft instead of once per pack. Two masks keep this honest:



(a) self-attention (`self.block`) card slots × card slots



(b) cross-attention (`self.future_tok`) card slots × history slots

Toy sizes: 4 wheel picks with 3 card slots each (really 18 with 13). Left: cards attend only to cards passed at the same pick. Right: a card passed at wheel pick t sees history slots $0..t$, *including* t (the drafter's own pick), and nothing later.

Figure 8: The pack decoder's masks. The real ones are 234×234 and 234×42.

One edge case: a few sets' files lack pack 1, pick 1, so that wheel pick has no passed cards. A token with nothing to attend to produces NaNs, so the code lets an all-empty pack attend to its first padding slot. Those outputs are never scored.

8 Step 6: from scores to probabilities

After the last block, a small head (layer norm, $192 \to 192$, GELU, $192 \to 1$) turns each passed-card token into a single number, its *logit*. The probability that card c comes back is the sigmoid of its logit:

$$P(c \text{ wheels} \mid \text{everything so far}) \;=\; \sigma(\text{logit}_c) \;=\; \frac{1}{1 + e^{-\text{logit}_c}}.$$

This is *not* a softmax. PickNet's question has exactly one answer per pack (the card the human took), so its probabilities sum to 1. Here several cards can come back, and each gets its own yes/no probability. The loss is the binary cross-entropy of each real passed card, averaged over all of them:

$$\text{loss} = -\frac{1}{N} \sum_{\text{passed cards}} \Big[\, y \,\ln p + (1 - y)\ln(1 - p) \,\Big].$$

A worked example. A real MSH test draft (from the QA section of `notebooks/wheel.ipynb`), pack 1, pick 2, drafted on day 23 of the format. The drafter was shown 13 cards and passed 12. Five come back at pick 10.

card passed	P(wheel)	came back?	loss (nats)	
Island	0.996	yes	0.004	
Hex Magic	0.877	yes	0.131	
Volcanic Villain	0.841	**no**	1.839	
Kingpin's Enforcers	0.573	yes	0.557	
Super Suit	0.495	yes	0.703	
Mockingbird, Ace Agent	0.315	no	0.378	
Wakandan Drone Flock	0.253	no	0.292	
Pet Avengers	0.240	no	0.274	
Beast, Erudite Aerialist	0.203	**yes**	1.595	
Jessica Jones, Private Eye	0.065	no	0.067	
Titania, Rugged Rumbler	0.047	no	0.048	
S.H.I.E.L.D. Spy Kit	0.044	no	0.045	
sum / mean	4.95	5	0.494	

- **Loss**: each card's loss is $-\ln p$ if it came back and $-\ln(1-p)$ if not. The two surprises, Volcanic Villain (taken at 84%) and Beast (back at 20%), cost 1.8 and 1.6 nats; the confident right calls cost almost nothing. This pack averages 0.494 nats per card; the model averages 0.422 over the MSH test set.
- **Expected count**: the probabilities sum to 4.95, and 5 cards came back. With 13 cards and 8 drafters, exactly $13 - 8 = 5$ cards return to this pick, but the model is never told that. Averaged over the test set the sum matches the true count at every pick (6.02 vs 6.00 at pick 1 … 0.99 vs 1.00 at pick 6), which is good evidence it has learned how a table behaves, not just per-card rates.
- **Calibration**: because the loss punishes confident mistakes, the probabilities can be read at face value. Of the second-best cards drafters pass that WheelNet gives 30–50%, a third come back.

9 The features-only variant: one model for many sets

The per-set model has a learned row for each card, so it can only score a set it was trained on. With `ids=False` the learned row goes away, and a card is *only* a function of its features:

$$\mathbf{e}_c \;=\; \mathrm{MLP}(\mathbf{f}_c), \qquad \mathrm{MLP} : 465 \to 384 \xrightarrow{\text{GELU}} 192.$$

$\mathbf{f}_c$ has 465 numbers: the 14 attributes above, card types and supertypes, Scryfall-derived rules features (power and toughness, keywords, mana production, rules-text roles), and a 384-number embedding of the rules text from the small language model `bge-small-en-v1.5`, as in PickNet. The features are standardised over all cards of all training sets. Because every card is described in the same terms, one set of weights scores any set, including cards it has never seen.

Everything else is unchanged, except that this variant uses the full draft context: rank (7 numbers), 17Lands skill buckets (4) and the two time numbers, 13 in all. The card table of the 8 training sets has 2,573 rows, and each set's local card ids are shifted into it (`remap()` in `train_wheelnet_multiset.py`). It has 4,051,009 parameters: the card MLP's 252,864 replace the per-set model's 67,968.

Trained on 20k drafts from each of 7 other sets plus MSH's own training drafts, it is the best MSH model (test log loss 0.4188, against 0.4218 for the shipped per-set model). Without MSH's drafts it scores 0.5245: it knows how tables behave but not which of a new set's cards are good.

10 Training

- **Data.** MSH's first 70% of drafts by time: 48,026 drafts, 9.1M passed cards. Each batch is 128 drafts, about 24,000 passed cards per update.
- **Loss.** Mean binary cross-entropy over every real passed card.
- **Optimiser.** AdamW (weight decay 0.01), gradients clipped at norm 1, a one-cycle learning-rate schedule: 5% linear warm-up to 10^{-3}, then a cosine decay to near zero.
- **Epochs.** 8 on MSH (3,008 steps, about 26 minutes on an Apple-silicon laptop). Other sets get round($8 \cdot$ 48k/training drafts), clamped to 3–8, so every set costs about the same.
- **Early stopping.** After each epoch the model is scored on the validation drafts (the next 15% by time); the best epoch's weights are kept. The test drafts (the last 15%) are scored once, at the end.
- **Regularisation.** Dropout 0.2 throughout. At 0.1 the model overfit after epoch 5.

Keeping it current. A model frozen after a format's first week falls behind a weekly retrain by 0.011 log loss by week 5, more than WheelNet's whole lead over the GBM baseline. Fine-tuning the previous week's model for 2 epochs on the newest week, at learning rate 3×10^{-4}, stays within about 0.001 of a full retrain (`scripts/temporal.py`).

11 Shapes and sizes

Tensor shapes through one forward pass, for a batch of $B = 128$ MSH drafts ($T = 42$ slots, 18 wheel picks, 13 passed-card slots, $d = 192$):

stage	shape	code
card table	`[340, 192]`	`card_table()`
pack bag, taken card, others bag, position	`[B, 42, 192]` each	`_bag(...)`, `taken`, `self.pos`
draft context	`[B, 1, 192]`, broadcast to all 42	`draft_ctx`
history tokens **x**	`[B, 42, 192]`	`h` before `self.hist`
history states **h**	`[B, 42, 192]`	`h` after `hist_norm`
wheel states	`[B, 18, 192]`	`h[:, self.wheel_pos]`
passed-card tokens **z**	`[B, 234, 192]`	`x`
(a) self-attention	block-diagonal, 13 slots per pack	`tgt_mask=self.block`
(b) cross-attention	234 queries per draft over 42 states	`memory_mask=self.future_tok`
logits	`[B, 3, 6, 13]`	`self.head(x)`

Where the 3.86M parameters are:

component	parameters	share
card table (339 card vectors + empty row, attribute projection)	67,968	2%
history encoder (4 transformer layers + final norm)	1,779,840	46%
pack decoder (3 blocks)	1,780,416	46%
token input layers (pack, taken, others, context, position)	156,864	4%
passed-card inputs (state projection, copies, wheel pick) and scoring head	78,913	2%
total	3,864,001	

Compared with PickNet, the card table is tiny (one set, 192 dimensions), and almost all the weights are in the two attention stacks.

12 Putting it together: one wheel pick, start to finish

Say we want the model's prediction for the cards a drafter passes at pack 2, pick 3 ($t = 16$) of MSH, on day 23 of the format.

1. **Look up card vectors** for every card that appears anywhere in the draft so far.
2. **Build tokens 0–16.** Token 16 = position 16 + bag(the 12 cards they were shown) + the card they took at this pick + context(day 23). Token 16 has no "others took" input yet (pack 2 hasn't come round), but tokens 8–13 do: they hold what the table took from each of round 1's first six packs.
3. **Run the history encoder.** Four layers of causal attention. State $\mathbf{h}_{16}$ now reflects all 17 picks so far, every pack they saw, and what wheeled in round 1.
4. **Build 11 passed-card tokens**, one per card left in the pack: card vector + copies + wheel-pick embedding for "pack 2, pick 3" + projected $\mathbf{h}_{16}$.
5. **Run the pack decoder.** In each of 3 blocks the 11 cards compare themselves with each other, consult $\mathbf{h}_0..\mathbf{h}_{16}$, and pass through an MLP.
6. **Score.** Eleven logits, eleven sigmoids: one probability per card that it is still there at pick 11. Exactly $14 - 10 = 4$ cards return, so their sum should be close to 4.

In training, steps 1–6 happen for all 18 wheel picks and all 128 drafts in the batch at once, and the losses from all real passed cards are averaged before each update.

13 How WheelNet differs from PickNet

The two models share a design and most of their code ideas, so the differences are easy to miss.

	PickNet	WheelNet
question	which card does the drafter take at t?	which passed cards come back at $t + 8$?
scored picks	all 45 slots	the 18 wheel-eligible slots
history token t holds	pack t, pick $t - 1$	pack t, pick t
decoder sees history	$\mathbf{h}_0..\mathbf{h}_t$ (before pick t)	$\mathbf{h}_0..\mathbf{h}_t$ (after pick t)
decoder tokens	every card in pack t	every card passed at t
output	softmax over the pack	independent sigmoid per card
loss	pick NLL + 0.3× deck BCE	BCE per passed card
extra heads	deck head	none
set embedding	yes (17 sets)	no (per-set); features-only variant for many sets
draft context	rank, skill, time, format, pack size	time only (shipped)
size	$d = 256$, 8 heads, 8.4M params	$d = 192$, 6 heads, 3.9M params
dropout	0.05	0.2

The one to watch if you combine them: PickNet's state at t has not seen pick t, and WheelNet's has. A wheel head on a PickNet trunk must add the pick at t itself, and must never attend to slot $t + 8$ or later, which holds the answer. `paper/combined/architecture.tex` describes such a shared-trunk version.

14 FAQ

Why not just use ALSA? ALSA is a per-card average over every pack a card appeared in. It knows nothing about what else is in your pack or what your table has been taking. On MSH, WheelNet cuts log loss by 8–9% against an ALSA-based model, and by 14% on the best card you pass, where ALSA is badly overconfident.

Does the model know who the other drafters are? No. It only sees which cards vanished from the packs that came back. It has to infer what the table is drafting from that.

Why one token per distinct card, not per copy? The label is "does at least one copy come back". Two copies of a common are one question, with a better chance of an answer of yes; the copies embedding tells the model there are two.

Can it score picks 7 and later? No. A pack passed at pick 7 or later of a 14-card round never returns with cards in it, so there is nothing to predict.

Does it know the drafter's skill? The shipped model doesn't, by design: rank and skill inputs didn't help, because what the model needs is how the *table* drafts, and the history already shows it.

Can it predict a set it has never seen? Only the features-only variant can, and not well: on day zero it is far behind an ALSA model fitted on a few days of the new set's data.